Architecture & High-Level Design (HLD)

1 Problem Statement

Business Problem: Automatically classify Amazon product reviews as positive or negative to help sellers understand customer sentiment at scale without manual reading.

1.0 ML Metric Targets

- Test Accuracy $\geq 85\%$
- ROC-AUC ≥ 0.90

1.0 Business Metric Targets

- Inference latency p95 $< 300\,\text{ms}$
- API availability $> 99\%$
- Data drift detection within 500 requests

1.0 Achieved

- Test Accuracy: 85.8%
- ROC-AUC: 0.936
- p95 latency: $< 200\,\text{ms}$

2 Architecture Overview

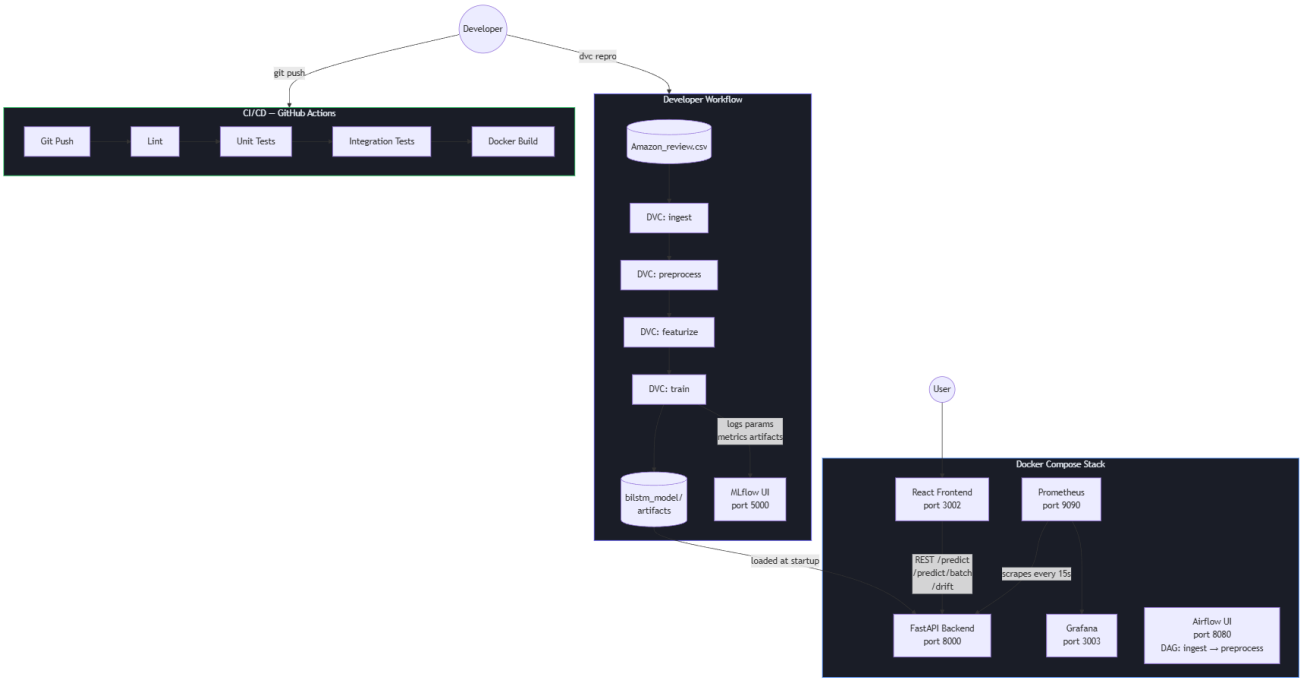

Figure 1: Architecture

3 Component Descriptions

3.1 Data Engineering Layer

Apache Airflow (`airflow/dags/sentiment_data_pipeline.py`)

- Orchestrates scheduled data pipeline runs
- DAG: `data_ingest` → `data_preprocess`
- Runs daily, supports manual trigger
- Handles retries, logging, and task dependency management

DVC (`dvc.yaml`)

- Versions all data artifacts and pipeline stages
- Full reproducible pipeline: `ingest` → `preprocess` → `featurize` → `train`
- `dvc repro` re-runs only changed stages
- Artifacts cached locally and pushable to remote storage

Airflow handles scheduling and orchestration of the data ingestion stages. DVC handles reproducibility, versioning, and the full ML pipeline including featurization and training. They are loosely coupled — Airflow feeds clean data; DVC takes it from there for ML-specific stages.

3.2 Data Pipeline Stages

Stage 1 — Ingest (`src/data/ingest.py`)

- Reads `Amazon_review.csv`
- Schema validation: checks required columns (`review`, `sentiment`) and valid values ({0, 1})
- Drops nulls and duplicates (4 found in 100,800 rows)
- Saves `Amazon_review_validated.csv`
- Logs: total rows, null counts, duplicate count, class distribution

Stage 2 — Preprocess (`src/data/preprocess.py`)

- Text cleaning: contraction expansion, URL removal, special character stripping, lowercasing
- Stopword removal and lemmatization via NLTK WordNetLemmatizer
- Train/val/test split: 70/10/20 stratified
- Computes baseline drift statistics (mean/std/min/max word length, vocab size)
- Saves: `train.csv`, `val.csv`, `test.csv`, `baseline_stats.json`

Stage 3 — Featurize (`src/features/featurize.py`)

- Fits `SimpleTokenizer` on training data only (prevents data leakage)
- Pads sequences to `max_length=128`
- Saves: `X_*.npy`, `y_*.npy`, `tokenizer.json`, `tokenizer.pkl`, `feature_meta.json`
- **Design decision:** Feature engineering is a separate DVC stage from text cleaning so tokenizer changes do not force re-cleaning of all text

Stage 4 — Train (`src/models/train.py`)

- Sets all random seeds for full reproducibility
- Builds and trains BiLSTM model
- Threshold tuning via precision-recall curve (optimal F1)
- Saves: `bilstm_model/`, `optimal_threshold.json`, `train_metrics.json`
- Logs everything to MLflow: params, per-epoch metrics, artifacts, git commit hash

3.3 Model Architecture

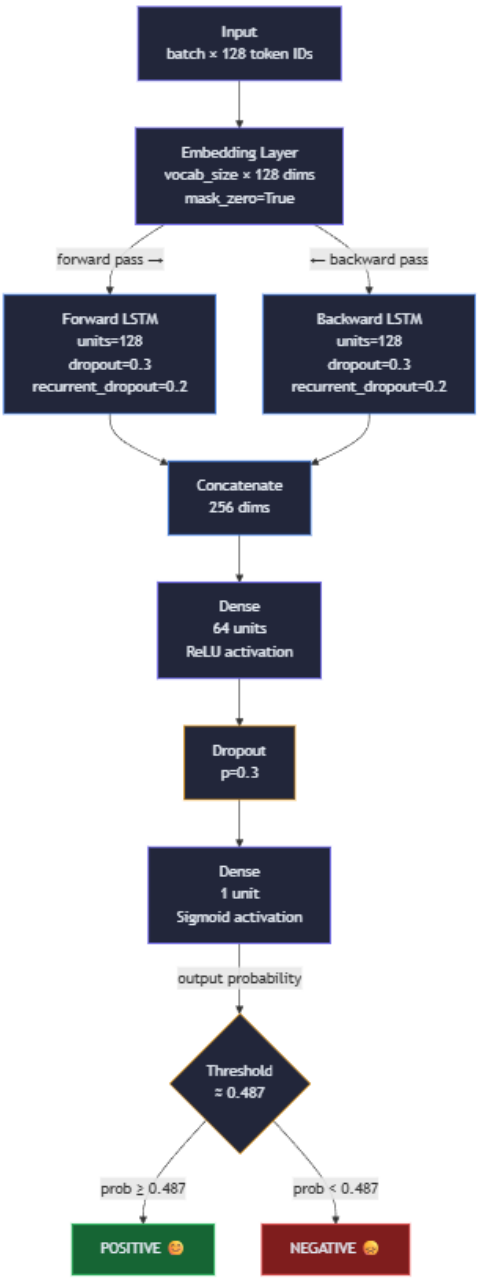

Figure 2: BiLSTM Model Architecture Diagram

Design Choices:

- **Bidirectional LSTM:** Captures both forward and backward context in reviews. A product review's sentiment can depend on words both before and after a key phrase.
- **mask_zero=True:** Tells the LSTM to ignore padding tokens, improving accuracy on variable-length inputs.
- **recurrent_dropout=0.2:** Regularizes recurrent connections specifically; reduces overfitting better than standard dropout alone.

- **ReduceLROnPlateau:** Halves learning rate when `val_loss` plateaus — extracts more accuracy in later epochs instead of stalling.
- **Threshold tuning:** Instead of fixed 0.5 threshold, finds optimal threshold on precision-recall curve to maximize F1.

3.4 Experiment Tracking (MLflow)

- **Backend:** SQLite (`mlruns/mlflow.db`) — fully on-device, no cloud required
- **Two experiments logged:** `BiLSTM-Base` and `BiLSTM-Deep` for comparison
- **Tracked per run:** all hyperparameters, per-epoch metrics, classification report, confusion matrix, model artifact, tokenizer, baseline stats, git commit hash, optimal threshold
- **Model Registry:** Best model registered as `SentimentBiLSTM` and transitioned to Production stage

3.5 Inference API (FastAPI)

- Loads model from local SavedModel path at startup
- Loads optimal threshold from `optimal_threshold.json`
- Initializes `DriftDetector` with baseline stats
- Applies identical preprocessing at inference time (clean → tokenize → pad)
- Exposes: `/predict`, `/predict/batch`, `/health`, `/ready`, `/drift`, `/metrics`
- Prometheus instrumented via `prometheus-fastapi-instrumentator`
- Custom metrics: prediction counters, latency histogram, positive ratio gauge, drift metrics

3.6 Drift Detection (`src/monitoring/drift_detector.py`)

- Maintains rolling window of 500 incoming review word counts
- Runs KS test comparing window distribution against training baseline
- Alert fires when p-value < 0.05 (statistically significant distribution shift)
- Results exposed via `/drift` endpoint and Prometheus metrics
- Grafana alert fires when positive prediction ratio goes outside 10%–90% range

3.7 Frontend (React + Vite)

- **Analyze page:** Single review input with example prompts, batch mode (one per line, max 50), prediction history sidebar
- **Pipeline page:** Live API status indicators, pipeline stage visualization, tech stack, links to all tool UIs
- Served via nginx in Docker; responsive design for mobile

3.8 Monitoring (Prometheus + Grafana)

Prometheus metrics tracked:

- `sentiment_predictions_total{sentiment}` — prediction counts by class
- `sentiment_prediction_latency_seconds` — latency histogram
- `sentiment_positive_ratio` — rolling positive class ratio
- `sentiment_input_length_words` — input text length distribution
- `sentiment_model_loaded` — model health gauge
- `sentiment_drift_detected` — KS test drift flag
- `sentiment_drift_ks_statistic` — KS test statistic
- `sentiment_drift_p_value` — KS test p-value
- `sentiment_drift_mean_shift` — mean word length shift from baseline
- `sentiment_drift_window_mean` — current window mean word length

Grafana: Pre-provisioned dashboard with 8 panels, auto-connects to Prometheus datasource on startup.

Alerts configured:

- `SentimentAPIDown` — fires if API unreachable for 1 minute (critical)
- `HighErrorRate` — fires if HTTP 5xx rate > 5% for 2 minutes (warning)
- `SentimentDriftDetected` — fires if positive ratio outside 10%–90% for 5 minutes (warning)
- `HighInferenceLatency` — fires if p99 latency > 500 ms for 3 minutes (warning)

3.9 Containerization (Docker Compose)

Six services in `docker-compose.yml`:

Table 1: Docker Compose Services

Service	Image	Port	Role
`backend`	Custom (`Dockerfile.backend`)	8000	FastAPI inference server
`frontend`	Custom (`Dockerfile.frontend + nginx`)	3002	React UI
`mlflow`	Custom (`Dockerfile.mlflow`)	5000	Experiment tracking UI
`prometheus`	`prom/prometheus:2.47.0`	9090	Metrics collection
`grafana`	`grafana/grafana:10.1.5`	3003	Metrics visualization
`airflow`	`apache/airflow:2.7.3`	8080	Pipeline orchestration

Loose coupling: Frontend communicates with backend only via REST API. All services communicate on the internal Docker network. Only required ports are exposed to the host.

4 Design Decisions Summary

Table 2: Key Design Decisions

Decision	Choice	Rationale
Model architecture	Bidirectional LSTM	Captures bidirectional context; better than unidirectional for sentiment
Tokenizer	Custom SimpleTokenizer	No TensorFlow dependency in data pipeline; Airflow-safe
Threshold	Tuned via PR curve	Maximizes F1 rather than using arbitrary 0.5
MLflow backend	SQLite	Fully on-device; no cloud or separate server required
API framework	FastAPI	Async support, Pydantic validation, auto OpenAPI docs
Serving	Direct SavedModel	Faster startup than MLflow pyfunc; model architecture is fixed
Drift detection	KS test	Non-parametric; works without assuming data distribution
Monitoring	Prometheus + Grafana	Industry standard; self-hosted; pre-built dashboard provisioning
CI/CD	GitHub Actions	Automatic on push; tests + lint + Docker build
Containerization	Docker Compose	Environment parity; loose coupling; single-command startup

5 Data Flow

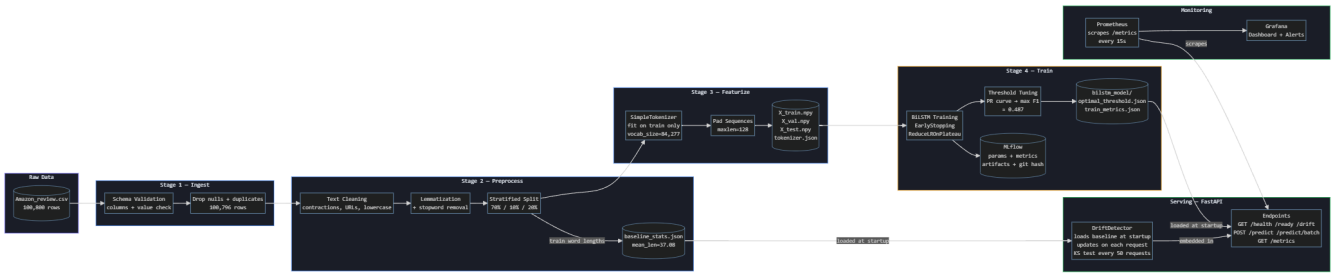

Figure 3: Data Flow

6 Security Considerations

- All containers communicate on internal Docker network
- Only required ports exposed to host machine
- Input validation via Pydantic schemas prevents malformed payloads
- No sensitive credentials committed to Git (`.env` file gitignored)
- Model artifacts gitignored; tracked via DVC

7 Known Limitations and Future Improvements

Table 3: Known Limitations and Planned Improvements

Limitation	**Future Improvement**
`sentiment_positive_ratio` tracks last batch only	Replace with rolling window ratio
No pretrained embeddings	Use GloVe or Word2Vec embeddings for better accuracy
KS test uses synthetic baseline samples	Store actual training word counts for precise KS test
No feedback loop for ground truth labels	Implement label collection for real-world performance tracking
SimpleTokenizer vocab differs from training tokenizer	Unify tokenizer across training and preprocessing